

	Pràctica Entregable 01 Classe per encriptar informació	IMO30FP01 REV 00 11/13
---	---	---------------------------------------

CURS:	DAM2	DATA:	Setembre-2026	Qualificació	
MÒDUL:	M0490 - Programació Serveis i Processos				
TEMA:	Enciptació i desencriptació de dades				
RA	RA5				
Professors	Cristina Reina - Carles Bonet				

Enunciat:

Formeu part de l'equip de desenvolupament d'una empresa que necessita **enviar petits missatges protegits entre diferents aplicacions**.

L'empresa no vol que el programa principal conegui els detalls de com es protegeixen els missatges. Per aquest motiu, **us demana crear una classe criptogràfica independent i reutilitzable**.

La classe haurà de permetre:

MISSATGE + CLAU → Procés d'ENCRIPATACIÓ → MISSATGE ENCRIPATAT

MISSATGE ENCRIPATAT + CLAU → Procés de DESENCRIPATACIÓ → MISSATGE ORIGINAL

La vostra missió és **dissenyar el vostre propi sistema de xifrat** i implementar-lo com una classe reutilitzable.

Un exemple per entendre com crear el vostre propi sistema:

Suposem que tenim:

- Missatge: HOLA
- Clau: 3

Podem utilitzar la clau per desplaçar cada lletra tres posicions:

- $H \rightarrow K$
- $O \rightarrow R$
- $L \rightarrow O$
- $A \rightarrow D$

Per tant: HOLA → KROD

Per recuperar el missatge original, fem el procés invers: KROD → HOLA.

1. Investiga el funcionament

Respon aquestes preguntes:

- Què passa si encriptem dues vegades el mateix missatge amb la mateixa clau?

Aleshores en missatge encriptat donat "KROD" l'hi donariem un altre cop 3 posicions i quedaria "NURG"

- Què passa si canviem la clau? Prova HOLA + clau 3 i HOLA + clau 7.

Amb clau = 3: HOLA → krod

Amb clau = 7:

Lletra	ASCII	+7	mod 256	Resultat
h	104	111	111	o
o	111	118	118	v
l	108	115	115	s
a	97	104	104	h

- Resultat Amb clau = 7 : HOLA → **ovsh**

- Què passa si intentem desencriptar utilitzant una clau incorrecta?

amb una clau incorrecta, el sistema no genera cap error ni avís simplement calcula igualment el resultat de la fórmula, però el text obtingut és incorrecte i no és el missatge original.

Com a mínim haurà de complir:

- El mateix missatge amb la mateixa clau ha de produir el mateix resultat. **SI**
- Una clau diferent ha de modificar el resultat. **SI**
- Si utilitzem la clau correcta, hem de poder recuperar exactament el missatge original. **SI**

Desencripta(Encripta("HOLA", "123"), "123") = "HOLA"

2. Dissenya l'algoritme propi d'enciptació i desenciptació

Abans de programar, heu de dissenyar el vostre sistema de xifrat.

Podeu utilitzar, per exemple:

- un sistema de substitució.
- un sistema de desplaçament.
- una combinació de diferents operacions.

La clau ha de participar obligatòriament en el procés.

Nom del vostre sistema	Desplaçament per Diferència Sumada (DDS)
Tipus de sistema	☐ Substitució ☒ <u>Desplaçament</u> ☐ Combinació ☐ Altres: _____
Tipus de clau	Alfanumèrica (paraula), es repeteix cíclicament fins a igualar la longitud del missatge
Com funciona l'enciptació?	Per cada caràcter del missatge, s'alinea amb el caràcter corresponent de la clau (repetida cíclicament). Es calcula la diferència dels codis: $\text{diferencia} = \text{lletra} - \text{clau}$. Aquesta diferència se suma a la lletra original i s'hi afegeix un desplaçament fix (OFFSET = 1000): $\text{xifrat} = \text{lletra} + \text{diferencia} + 1000$. Com que $\text{lletra} + \text{diferencia} = 2 \cdot \text{lletra} - \text{clau}$ pot ser negatiu, l'offset desplaça tota l'escala cap amunt perquè el resultat sigui sempre un codi vàlid.

Com funciona la descriptació?	Es resta l'offset i s'aïlla la lletra original: $\text{lletra} = (\text{xifrat} - 1000 + \text{clau}) / 2$. La divisió és sempre exacta perquè, per construcció, $\text{xifrat} - 1000 + \text{clau}$ és el doble del codi de la lletra original: $(2 \cdot \text{lletra} - \text{clau} + 1000) - 1000 + \text{clau} = 2 \cdot \text{lletra}$.
Com sé que el meu sistema funciona?	S'ha comprovat manualment amb HOLA i clau PEP (exercici 3) i amb el programa amb HOLA MÓN, que inclou un espai i una Ó. Les versions anteriors fallaven: amb valor absolut i amb la resta no es podia recuperar la O. Amb % 256 fallava la Ó, perquè $2 \cdot 211 - 80 = 342$ i $342 \% 256 = 86$ es desxifrava com a S. El mòdul esborra quantes voltes s'han fet. L'offset només desplaça i no esborra informació, per això el desxifrat és exacte. Limitació: només funciona per a caràcters fins a aproximadament U+7E00 (els emojis no).

3. Demuestra el teu algoritme

Abans de començar a programar, feu un exemple manual complet.

Missatge: HOLA

Clau: PEP (repetida cíclicament: P-E-P-P)

X sent la diferència

<u>Lletra</u>	<u>ASCII</u> <u>I</u>	<u>Clau</u>	<u>x = lletra - clau</u>	<u>xifrat =</u> <u>lletra + x</u>	<u>+ 1000</u>	<u>Xifrada</u>
H	72	P = 80	-8	64	1064	Ш
O	79	E = 69	10	89	1089	c
L	76	P = 80	-4	72	1072	a
A	65	P = 80	-15	50	1050	K

Encriptació: H → Ш, O → c, L → a, A → K

Resultat: **ШcaK**

Igualment, descripta el resultat i demostra que recuperes HOLA.

<u>Caràcter xifrat</u>	<u>Codi</u>	<u>Clau</u>	<u>$(\text{xifrat} - 1000 + \text{clau}) / 2$</u>	<u>Resultat</u>
Ш	1064	P = 80	$(1064 - 1000 + 80) / 2 = 72$	H
c	1089	E = 69	$(1089 - 1000 + 69) / 2 = 79$	O
a	1072	P = 80	$(1072 - 1000 + 80) / 2 = 76$	L
K	1050	P = 80	$(1050 - 1000 + 80) / 2 = 65$	A

4. Implementació de la classe “Encripta” en Java

Crea una classe independent anomenada:

ClasseCriptografica

La classe haurà d'oferir, com a mínim, aquests dos mètodes:

- `Encripta(Missatge, Clau)`
- `Desencripta(Missatge, Clau)`

Els mètodes han de retornar el missatge resultant.

La classe criptogràfica no ha de contenir el programa principal. El programa principal serà qui utilitzarà la classe.

Programa principal → ClasseCriptografica → Encripta / Desencripta

5. Proves del sistema dissenyat d'encriptació

Test	Prova	Resultat
Test 1	Missatge curt: HOLA	☐ Correcte ☐ Incorrecte Observacions: El missatge HOLA queda xifrat com a ШcaK , que correspon als codis 1064, 1089, 1072 i 1050. Cada lletra s'ha xifrat amb la lletra de la clau que li toca (P-E-P-P). Per exemple, la H: $2 \cdot 72 - 80 + 1000 = 1064$ (Ш). En desxifrar, $(\text{xifrat} - 1000 + \text{clau}) / 2$ retorna els codis 72, 79, 76 i 65, és a dir, HOLA, idèntic a l'original.

Test 2	Missatge amb espais: HOLA MÓN	☐ Correcte ☐ Incorrecte Observacions: Codis xifrats: 1064, 1089, 1072, 1050, 995, 1074, 1342, 1087. Es recupera HOLA MÓN, Codis xifrats: 1064, 1089, 1072, 1050, 995, 1074, 1342, 1087. Es recupera HOLA MÓN, amb l'espai i la Ó. Amb la versió anterior (% 256) la Ó es recuperava com a S. L'offset ho arregla perquè no s'esborra informació.
Test 3	Missatge més llarg: almenys 30 caràcters	☐ Correcte ☐ Incorrecte Observacions: Missatge: La criptografia protegeix els missatges més secrets. Xifrat: аєQуЖ0уЙЎАњЄvњЅ ЌЎЎЎЎАжЖуЎΨQΘv ЌЌЄЅАжЌQЌdЌ щЌЎжЌЎЙЌΘ

		El primer espai (32) amb P dona 984 (Q). Les lletres amb accent (é), els espais i el punt final es recuperen sense errors, i el text desxifrat és idèntic a l'original. Això confirma que el sistema no depèn de la longitud del missatge.
Test 4	Clau diferent: comprovar que el resultat canvia	☐ Correcte ☐ Incorrecte Observacions: PEP ШcaK HOLA ABC зфнЩ HOLA 123 чєэЙ HOLA El mateix missatge dona tres resultats diferents segons la clau, així que la clau participa realment en el procés. Per exemple, la H (72) amb P (80) dona 1064, amb A (65) dona 1079 i amb 1 (49) dona 1095. Cada unitat que baixa el codi de la clau fa pujar el xifrat una unitat. El mateix missatge amb la mateixa clau dona sempre el mateix resultat (determinista), i amb la clau correcta sempre es recupera HOLA.
Test 5	Encriptar i desencriptar: recuperar exactament el	☐ Correcte ☐ Incorrecte Observacions:

	missatge original Àçñ€ ¿Ç? amb clau PEP dona el xifrat æðϣ堇 ϣR□C (codis 1304, 1393, 1402, 17648, 995, 1302, 1318, 1057), i es recupera Àçñ€ ¿Ç?.	Tots els missatges dels tests 1-3 i el de símbols es recuperen exactament. L'euro (8364) és el cas més extrem: $2 \cdot 8364 - 80 +$ $1000 = 17648$ (堇), que encara cap en un char. La divisió entre 2 és sempre exacta perquè xifrat $- 1000 +$ clau és per construcció 2·lletra. Limitació: Un char de Java arriba com a màxim a 65535. Com que el xifrat duplica el codi, els caràcters per sobre d'uns 32.000 (aprox. U+7E00), com els emojis, se surten del rang i no es recuperen bé. Per a text normal en català, castellà o anglès no hi ha cap problema.
Test 6	Clau incorrecta: explicar què passa	☐ Correcte ☐ Incorrecte Observacions: El sistema no llança cap error ni avís: aplica la fórmula igualment i retorna un text incorrecte. Per què surt @ME9? H: $(64 + 65) / 2 = 64$ (@) O: $(89 + 66) / 2 = 77$ (M)

		L: $(72 + 67) / 2 = 69$ (E) A: $(50 + 65) / 2 = 57$ (9) En general, amb una clau errònia es recupera lletra + $(\text{clau_falsa} - \text{clau_correcta}) / 2$, que no és la lletra original. Amb XYZ el resultat és LYQE. Curiositat: Amb la clau PEQ, que només difereix en un caràcter, el resultat és HOLA. La divisió entera trunca ($76,5 \rightarrow 76$) i absorbeix la diferència d'una unitat. Això mostra que el sistema és didàctic i no segur, perquè una clau gairebé correcta pot revelar el missatge.
--	--	---

6. Implementació d'un sistema de xifrat AES en Java

Fins ara heu dissenyat i implementat el vostre propi sistema de xifrat. Ara fareu un pas més: **utilitzareu AES (Advanced Encryption Standard)**, un algoritme de xifrat simètric utilitzat en sistemes reals.

En aquesta part **no haureu de crear l'algoritme AES des de zero**. Java proporciona una API de criptografia que permet utilitzar-lo.

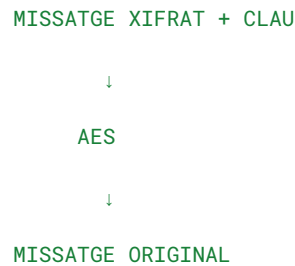
Objectiu

Crear una classe anomenada **ClasseAES** que permeti:

MISSATGE + CLAU



i recuperar posteriorment el missatge original:



La classe haurà d'oferir, com a mínim, els dos mètodes següents:

```
public static String encripta(String missatge, String clau)
public static String desencripta(String missatgeXifrat, String clau)
```

6.1. Eines disponibles

Per realitzar aquesta activitat podeu utilitzar les classes i mètodes següents de Java:

Classes:

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
```

Mètodes i elements que podeu consultar i utilitzar:

```
getBytes()
Cipher.getInstance()
cipher.init()
cipher.doFinal()
```

`Base64.getEncoder()`

`Base64.getDecoder()`

`SecretKeySpec`

`Cipher.ENCRYPT_MODE`

`Cipher.DECRYPT_MODE`

Aquestes són les eines que necessitareu per construir el vostre sistema.

No es proporcionarà el codi complet de la classe. Haureu d'investigar el funcionament de les classes i mètodes proporcionats i decidir com combinar-los per construir els processos d'enciptació i desenciptació

Trobareu un [annex explicatiu](#) breu al final del document.

S'utilitza l'API estàndard de Java (`javax.crypto`) en lloc d'un algoritme propi. La classe `ClasseAES` és independent i reutilitzable: no conté cap `main`, i el programa principal només crida a dos mètodes estàtics.

Programa principal → `ClasseAES` → encripta / descripta

Mètode	Entrada	Sortida
<code>encripta(String missatge, String clau)</code>	text pla + clau	missatge xifrat en Base64
<code>descripta(String missatgeXifrat, String clau)</code>	Base64 + clau	text pla original

Decisions de disseny:

- Algorisme: `Cipher.getInstance("AES")`, que a Java equival a `AES/ECB/PKCS5Padding`.
- Clau: els bytes UTF-8 del text de la clau, que ha de fer 16, 24 o 32 bytes (`AES-128`, `192` o `256`).
- Codificació: el text es converteix a bytes amb UTF-8 perquè es recuperi exactament (accents inclosos).
- Sortida: Base64, perquè el xifrat són bytes arbitraris que no es poden mostrar com a text.

6.2. Implementa la classe **ClasseAES**

Crea una classe independent anomenada **ClasseAES**, que tingui els mètodes:

encripta (missatge, clau)

desencripta (missatge, clau)

El mètode **encripta()** haurà de:

1. Rebre un missatge i una clau.
2. Preparar la clau perquè pugui ser utilitzada per AES.
3. Crear i configurar el sistema de xifrat.
4. Convertir el missatge al format necessari.
5. Xifrar el missatge.
6. Convertir el resultat en un format que es pugui retornar com a **String**.
7. Retornar el missatge xifrat.

El mètode **desencripta()** haurà de realitzar el procés invers:

1. Rebre el missatge xifrat i la clau.
2. Preparar la clau.
3. Crear i configurar el sistema de desxifrat.
4. Recuperar les dades xifrades.
5. Desxifrar-les.
6. Convertir el resultat novament a text.
7. Retornar el missatge original.

encripta()

1. Crea un **SecretKeySpec** a partir dels bytes de la clau i l'algorisme "AES".
2. Crea el **Cipher** amb **Cipher.getInstance("AES")**.
3. L'inicialitza amb **cipher.init(Cipher.ENCRYPT_MODE, clauAES)**.
4. Converteix el missatge a bytes UTF-8 i el xifra amb **cipher.doFinal(...)**.
5. Codifica el resultat amb **Base64.getEncoder().encodeToString(...)**.
6. Retorna el **String** xifrat. **desencripta()** fa el procés invers: prepara la clau, inicialitza el **Cipher** amb **DECRYPT_MODE**, decodifica el Base64 amb **Base64.getDecoder().decode(...)**, aplica **doFinal(...)** i converteix els bytes a **String** amb UTF-8.

6.3. Programa principal

Crea un `ProgramaPrincipal` que utilitzi la classe `ClasseAES`.

Utilitza inicialment les dades següents:

`Missatge: Aquest és un missatge secret.`

`Clau: 1234567890123456`

El programa haurà de:

1. Mostrar el missatge original.
2. Encriptar-lo utilitzant AES.
3. Mostrar el missatge xifrat.
4. Desencriptar el resultat utilitzant la mateixa clau.
5. Mostrar el missatge recuperat.

El resultat final ha de demostrar que:

`Desencripta(Encripta(missatge, clau), clau) = missatge`

Dades inicials:

- Missatge: Aquest és un missatge secret.
- Clau: 1234567890123456 (16 caràcters = 128 bits) Resultat de l'execució:

Missatge original: Aquest és un missatge secret.

Missatge xifrat: 3UXPE1odbnoSr4WWreU9QCSMsetZu7rBCYtwVkl6fJc=

Missatge recuperat: Aquest és un missatge secret.

Es compleix `Desencripta(Encripta(missatge, clau), clau) = missatge`.

Nota: si la consola no està en UTF-8, la `e` pot sortir com a `é`. És només un problema de visualització, perquè el xifrat i el desxifrat treballen amb bytes UTF-8. Es pot solucionar amb `java -Dfile.encoding=UTF-8`.

6.4. Experimenta amb el teu programa

Una vegada el programa funcioni, realitza les proves següents:

Prova 1. Clau correcta

Comprova que el missatge original es recupera correctament.

Prova 2. Clau diferent

Intenta desenscriptar el missatge utilitzant una clau diferent.

Explica què passa.

Prova 3. Missatge diferent

Utilitza un altre missatge i comprova que el procés continua funcionant.

Prova 4. Clau de longitud incorrecta

Prova d'utilitzar una clau que no tingui la longitud adequada.

Observa què succeeix i explica el motiu.

Prova	Entrada	Resultat
1. Clau correcta	1234567890123456	Es recupera Aquest és un missatge secret.
2. Clau diferent	6543210987654321	BadPaddingException: Given final block not properly padded
3. Missatge diferent	Un altre missatge, amb accents: Ó, ç, à!	Es recupera idèntic
4. Clau de longitud incorrecta	1234567890 (10 bytes)	InvalidKeyException: Invalid AES key length: 10 bytes

Prova 1. El missatge es recupera exactament, accent inclòs.

Prova 2. Amb una clau errònia el desxifrat produeix bytes sense sentit i l'últim bloc no acaba amb un padding PKCS5 vàlid, de manera que Java llança una excepció. Això és diferent del sistema propi (DDS), que retornava text incorrecte sense cap avís. Cal tenir en compte que, per casualitat (aprox. 1 de cada 256 vegades), el padding pot resultar vàlid i llavors es retorna text brossa sense error. Per tant, l'excepció no és una garantia de detecció de clau errònia.

Prova 3. Xifrat obtingut:

LcAL2/QtKaxAOfiTGOOxEu6F7fVjeDP4xBOKNg2d0UBZaJ/DI8w4rSpLVFUeK0M6.
Es recuperen correctament Ó, ç i à. Com que es treballa amb bytes UTF-8, no hi ha
els problemes de rang de caràcters del sistema propi.

Prova 4. AES només accepta claus de 16, 24 o 32 bytes (128, 192 o 256 bits). L'error
es produeix a cipher.init(), abans de xifrar res.

6.5. Comprèn el codi

No n'hi ha prou amb aconseguir que el programa funcioni. Has de poder explicar què està fent.

Respon les preguntes següents:

1. **Quina classe de Java s'encarrega del xifrat?** `javax.crypto.Cipher`.
2. **Per a què serveix `SecretKeySpec`?** Converteix els bytes de la clau en un objecte de clau secreta que Cipher pot utilitzar, i n'indica l'algorisme ("AES").
3. **Què fa `Cipher.getInstance()`?** Crea l'objecte xifrador per a l'algorisme indicat. Amb "AES" s'usen per defecte el mode ECB i el padding PKCS5.
4. **Diferència entre `ENCRYPT_MODE` i `DECRYPT_MODE`?** S'indiquen a `cipher.init()` i determinen si el Cipher xifra o desxifra. Amb la mateixa clau, el mode decideix el sentit de l'operació.
5. **Per a què serveix `doFinal()`?** Processa les dades, aplica (o treu) el padding i retorna el resultat com a array de bytes.
6. **Per què convertim el missatge a bytes?** AES treballa sobre bytes, no sobre text. `getBytes(UTF_8)` dona una representació única i recuperable del text.
7. **Per què Base64?** El resultat de `doFinal()` són bytes arbitraris, no imprimibles. Base64 els codifica amb caràcters imprimibles (A-Z a-z 0-9 + / =) i és reversible sense pèrdua.
8. **Quina mida té la clau?** 1234567890123456 són 16 caràcters = 16 bytes = 128 bits (AES-128).
9. **Què vol dir que AES sigui simètric?** Que la mateixa clau serveix per xifrar i per desxifrar.
10. **Per què necessitem la mateixa clau?** Perquè el desxifrat només desfà el que ha fet el xifrat amb aquella clau. Amb una altra clau el resultat és brossa o un error (prova 2). **Mida del xifrat.** El missatge ocupa 30 bytes (la e en ocupa 2 en UTF-8). AES xifra en blocs de 16 bytes, així que el padding l'omple fins a 32 bytes, que en Base64 són 44 caràcters. Per això el resultat acaba en =.

Finalment, respon:

Per què és preferible utilitzar un algoritme criptogràfic estàndard com AES en lloc de crear el nostre propi sistema de xifrat?

- **Seguretat provada.** AES és un estàndard analitzat durant anys per la comunitat criptogràfica. El sistema propi (DDS) només fa una operació aritmètica per caràcter: qui conegui un parell missatge/xifrat obté la clau amb una resta ($\text{clau} = 2 \cdot \text{lletra} + 1000 - \text{xifrat}$).
- **Errors de disseny.** El sistema propi va fallar amb % 256, no admet caràcters per sobre d'aprox. U+7E00 i una clau gairebé correcta pot revelar el missatge (cas PEQ).
- **Robustesa i detecció d'errors.** AES avisa quan la clau o la longitud no són vàlides; el sistema propi retorna text incorrecte sense cap avís.
- **Eficiència i suport.** AES està optimitzat a la llibreria de Java i té suport de maquinari a molts processadors.
- **Reutilització.** És un estàndard interoperable entre sistemes i llenguatges.

Limitacions d'aquesta implementació

Cipher.getInstance("AES") utilitza el mode **ECB**, on blocs iguals donen criptogrames iguals i el mateix missatge amb la mateixa clau dona sempre el mateix xifrat. Per a un ús real caldria **CBC o GCM amb un IV** (tal com explica l'annex), i derivar la clau d'una passphrase amb salt en lloc d'usar-la directament.

Lliurament

Heu de lliurar en un repositori GitHub:

1. Sistema de xifrat propi

- `ClasseCriptografica.java`
- `ProgramaPrincipal.java`

2. Implementació AES

- `ClasseAES.java`
- `ProgramaPrincipalAES.java`

3. Documentació

- Document PDF amb les respostes i explicacions de la pràctica.

Cal lliurar aquesta documentació en un Repositori GitHub i que el compartiu amb la Cristina i en Carles.

Xifrat AES

El xifrat AES (Advanced Encryption Standard) va ser aprovat com estàndard de xifrat pel govern dels EUA després d'un procés de 5 anys. El xifrat va ser desenvolupat per dos criptòlegs belgues J.Daemen i V. Rijmen que el van enviar amb el nom de Rijndael.

Característiques:

- Xifrat de bloc: mida 128 bits.
- Clau de xifrat de 128,192 o 256 bits.

Actualment es considera un sistema de xifrat segur (la NSA exigeix clau de 192 o 256 bits per documents d'alta seguretat). És un algoritme ràpid, AES 256 és més ràpid que DES i per tant molt més ràpid que la solució 3DES.

Elements del xifrat:

- IV (Vector inicialització): És necessari per iniciar cada procés de xifrat. Té una mida de 16 bytes (1 bloc). El seu objectiu és evitar que dos blocs iguals tinguin el mateix criptograma.
- Clau (Key): És la principal informació per xifrar i desxifrar en els algoritmes simètrics. A la implementació per defecte de .NET s'utilitza la de 256 bytes.

Cipher mode:

ECB: xifra cada bloc per separat. Per tant, blocs idèntics donen el mateix criptograma.

CBC: s'utilitza un vector d'inicialització (IV) i cada bloc es combina amb el bloc anterior amb una operació XOR. Això assegura que blocs iguals no donin un criptograma idèntic.

Clau de xifrat:

Existeixen mètodes per obtenir la clau de forma automàtica, però això només té sentit si aquesta clau es pot enviar de forma segura. Volem obtenir la clau a partir de passphrase que introdueixi l'usuari.

La forma més còmoda per l'usuari és introduir una password o passphrase enlloc de una clau i un vector d'inicialització. Utilitzem `Rfc2898DeriveBytes` per obtenir un *hash* de la combinació dels dos textos. S'afegeix una cadena *salt* per fer més difícil l'atac per RainBow Tables. L'algoritme es pot aplicar iterativament per augmentar la complexitat.

Rúbrica:

Criteri	Què s'avalua	Punts	
1. Disseny del sistema propi	Defineix l'algoritme, la clau i explica correctament encriptació i desencriptació	1,5	
2. Implementació del sistema propi	ClasseCriptografica funciona correctament i està separada del programa principal	1,5	
3. Ús de la clau	La clau participa realment en el procés i una clau incorrecta no permet recuperar correctament el missatge	1,0	
4. Proves del sistema propi	Realitza i documenta correctament els tests proposats	1,0	
5. Implementació d'AES	Utilitza correctament l'API de Java i implementa ClasseAES amb encriptació i desencriptació	2,0	
6. Comprensió de l'API criptogràfica	Explica Cipher, SecretKeySpec, init(), doFinal(), Base64, etc.	1,0	
7. Experimentació amb AES	Realitza les proves amb diferents claus i missatges i explica els resultats	0,75	
8. Documentació i comparació	Respon les preguntes i compara el sistema propi amb AES de manera raonada	0,75	
9. Organització i lliurament	Estructura correcta del projecte, noms de classes, repositori GitHub i documentació	0,5	
TOTAL		10	